Huffman’s coding
aka
Compact coding
aka
Optimal coding

Coding of symbols
(aka source coding)

Fixed length
Coding
(assumes
uniform PDF)

Variable length
Coding
(assumes
Non-uniform PDF)

Example – 1 (Uniform PDF-fixed length coding)

	Probability
S_1	0.25
S_2	0.25
S_3	0.25
S_4	0.25

	Code
S_1	00
S_2	01
S_3	01
S_4	11

- 4 symbols: Log(M) = 2 bits
- Length of S_1 = 1; Length of S_2 = 1; Length of S_3 = 2; and Length of S_4 = 2
- Average = $P(S_1)$*Length of S_2 + $P(S_2)$*Length of S_2 + $P(S_3)$*Length of S_3 + $P(S_4)$*Length of S_4
  = 0.25*2 + 0.25*2 + 0.25*2 + 0.25*2 = 2 bits

Example – 2 (non-uniform PDF-fixed length coding)

	Probability
S_1	0.5
S_2	0.25
S_3	0.125
S_4	0.125

	Code
S_1	00
S_2	01
S_3	01
S_4	11

- 4 symbols: Log(M) = 2 bits
- Length of S_1 = 1; Length of S_2 = 1; Length of S_3 = 2; and Length of S_4 = 2
- Average = $P(S_1)$*Length of S_2 + $P(S_2)$*Length of S_2 + $P(S_3)$*Length of S_3 + $P(S_4)$*Length of S_4

= 0.5*2 + 0.25*2 + 0.125*2 + 0.125*2 = 2 bits

Example – 3 (non-uniform PDF-variable length coding)

	Probability
S_1	0.5
S_2	0.25
S_3	0.125
S_4	0.125

	Code
S_1	0
S_2	10
S_3	110
S_4	111

- 4 symbols: Log(M) = 2 bits
- Length of S_1 = 1; Length of S_2 = 2; Length of S_3 = 3; and Length of S_4 = 3
- Average = $P(S_1)$*Length of S_2 + $P(S_2)$*Length of S_2 + $P(S_3)$*Length of S_3 + $P(S_4)$*Length of S_4

= 0.5*1 + 0.25*2 + 0.125*3 + 0.125*3 = 1.75 bits

Observations from examples 1,2, & 3

- Example 1 – uniform PDF
- PDF of Example 2 = PDF of Example 3 = non-uniform pdf
- Entropy of example 1 –
- Entropy of example 2 or 3 -

Shannon's first theorem : Source-coding theorem

- Given a dms of entropy H(*S*), the average code-word length $\overline{L}$ for any source coding is

$$\overline{L} \geq H(S)$$

i.e.) $L_{\min} = H(S) \quad \& \quad \eta = \frac{H(S)}{\overline{L}}$

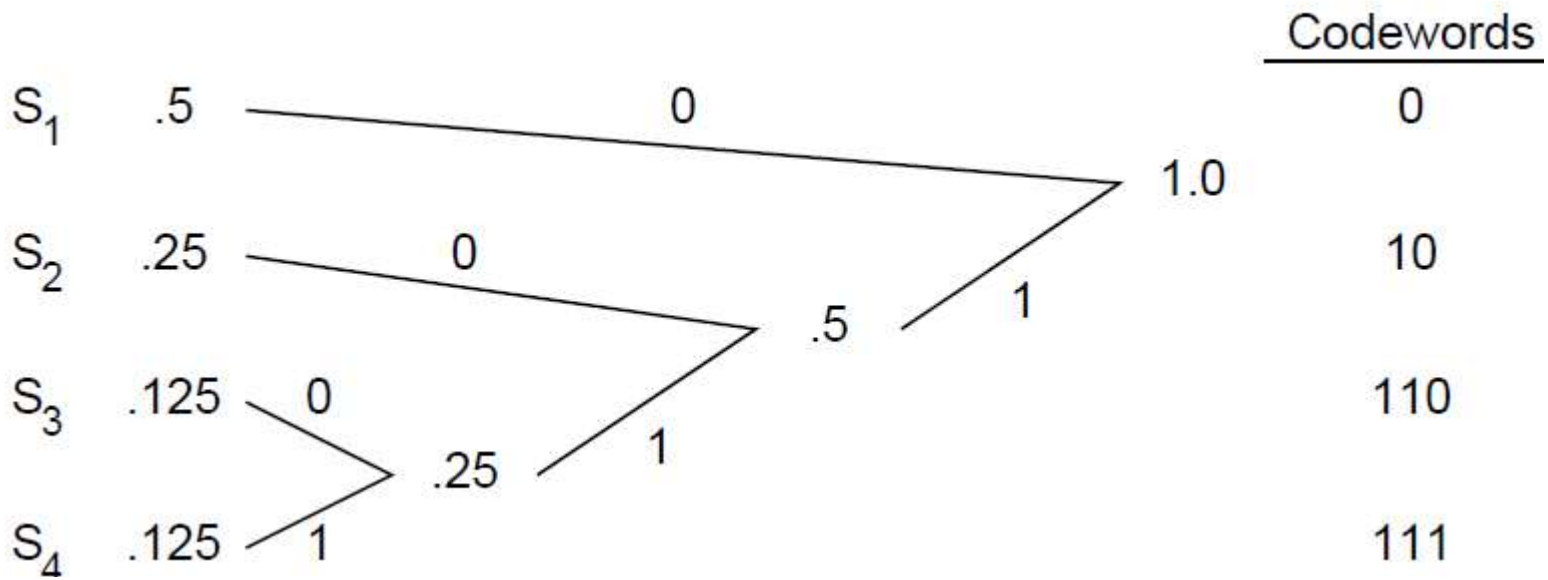
Huffman coding

- Variable length coding
- Does not assume uniform distribution
- Uses the PDF information efficiently

Huffman coding-example

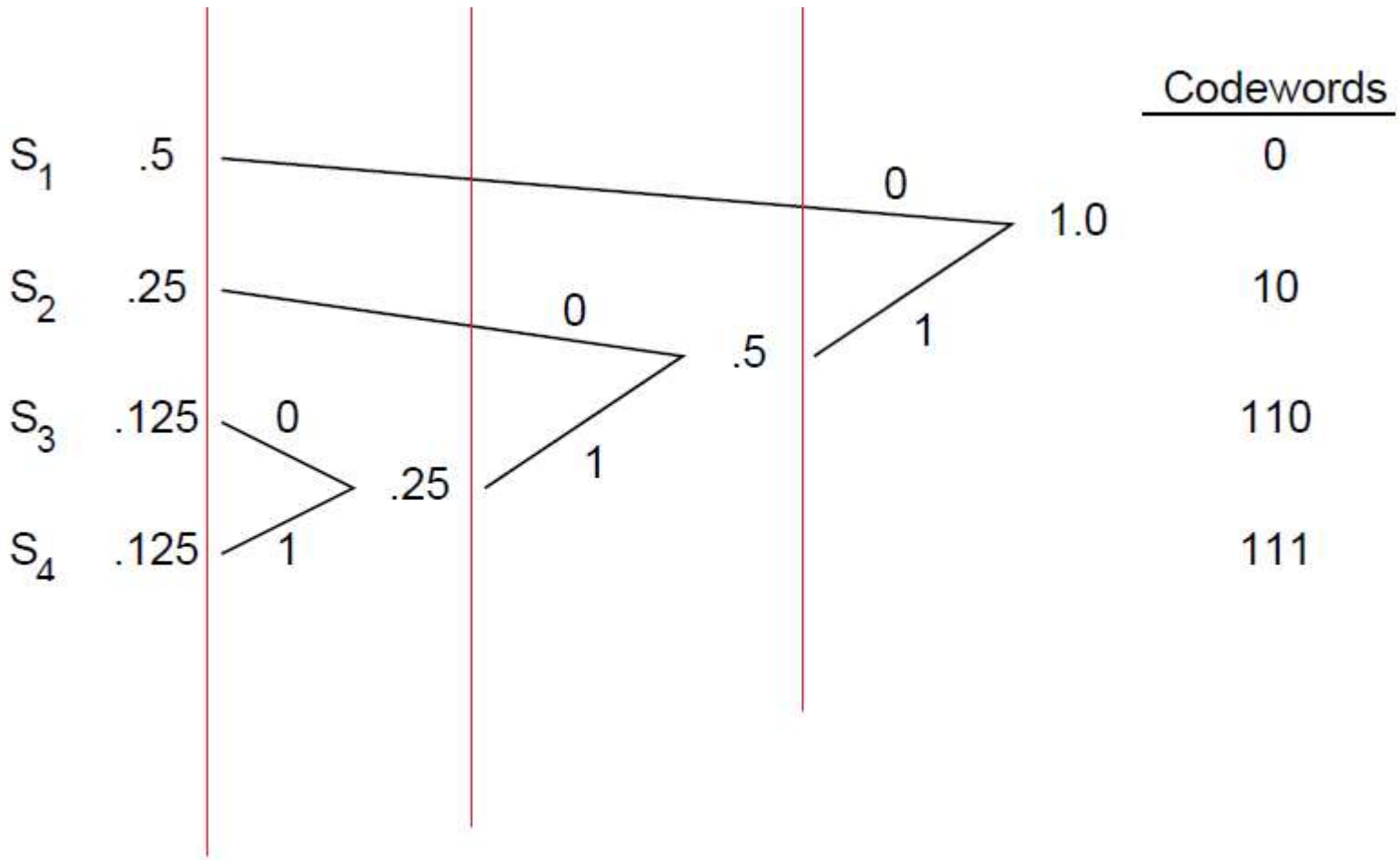
- 4 symbols – S_1, S_2, S_3, S_4
- PDF: 0.5, 0.25, 0.125, 0.125 respectively

Huffman code with 4 symbols



$$\text{average length} = (1)\left(\frac{1}{2}\right) + (2)\left(\frac{1}{4}\right) + (3)\left(\frac{1}{8}\right) + (3)\left(\frac{1}{8}\right)$$
$$= 1.75 \text{ bits/symbol}$$

4 symbols with fixed length code: average length = 2



there are 3 stages - @ every stage order the probability values

Observations

- Variable length codes
- Lower probability is assigned lengthier code

- Efficient compared to fixed length code
- For the previous e.g.
- Fixed length code gives an average length = 2

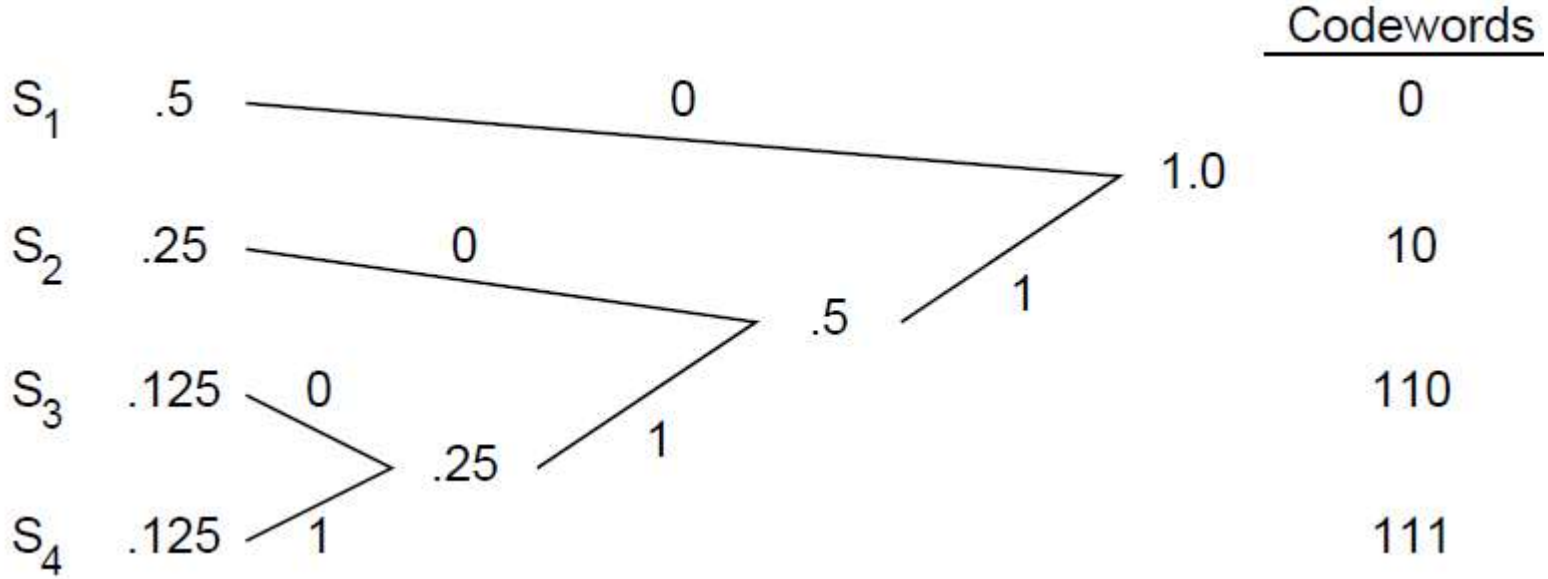
What is the entropy?

- Probabilities of

  $S_1 = ½$, $S_2 = ¼$, $S_3 = 1/8$, $S_4 = 1/8$

$$
\begin{aligned}
H &= \frac{1}{2}\log_2 2 + \frac{1}{4}\log_2 4 + \frac{1}{8}\log_2 8 + \frac{1}{8}\log_2 8 \\
&= \frac{1}{2} + \frac{1}{2} + \frac{3}{8} + \frac{3}{8} \\
&= 1.75
\end{aligned}
$$

Uniquely decodable



- $S_2\ S_1\ S_1\ S_3\ S_4\ S_1\ S_1\ S_4\ S_2$ ….
- 10001101110011110 …

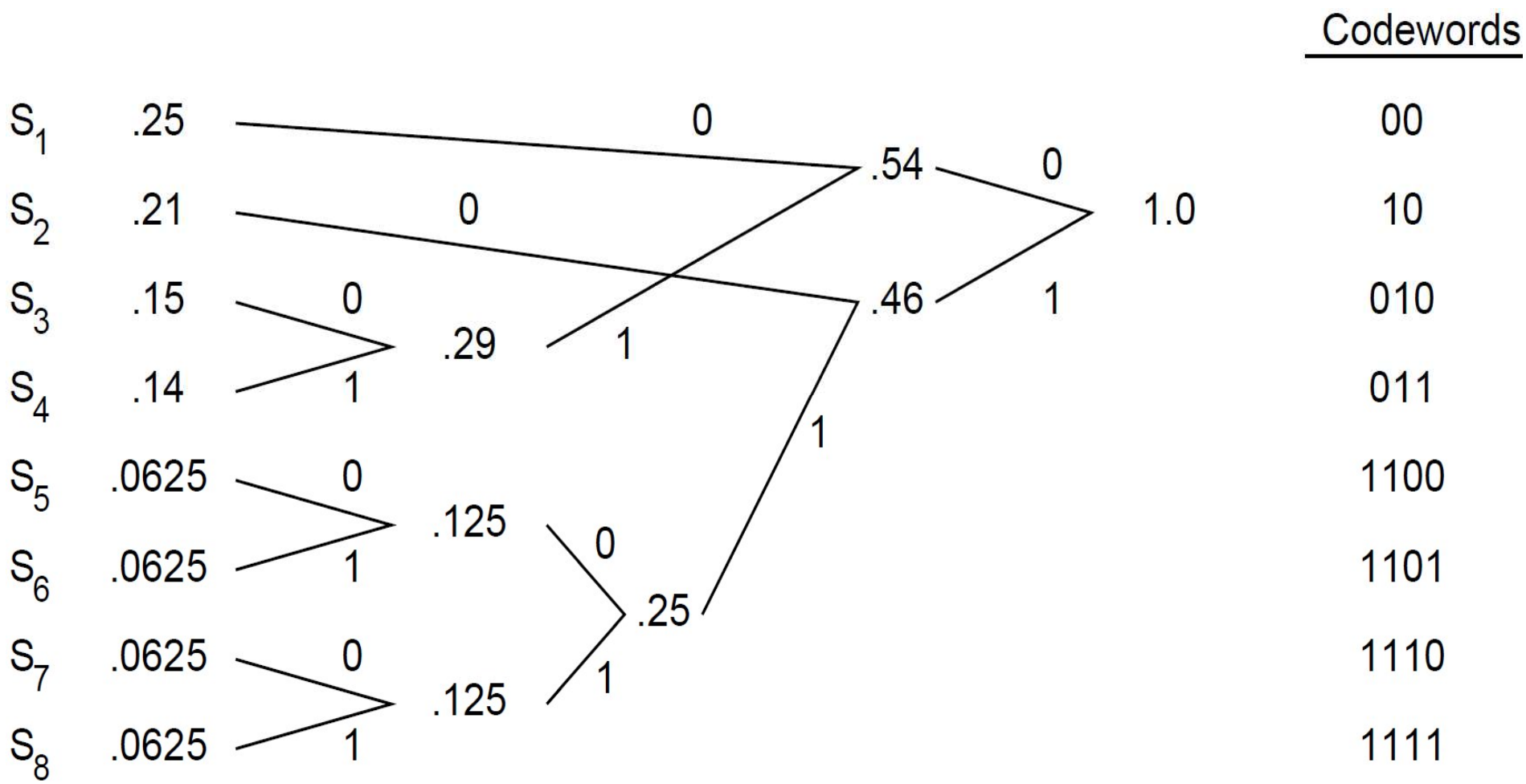
If 0
Take decision
Else
Wait for the next bit
end

Constraint
Maximum of 3 bits

One more e.g.

- Source symbol PDF: {0.25, 0.21, 0.15, 0.14, 0.0625, 0.0625, 0.0625, 0.0625 }

Another example



Average length and entropy – previous example

$$\begin{aligned}\text{average length} &= (2)(0.25+0.21)+(3)(0.15+0.14) \\ &\quad +(4)(0.0625+0.0625+0.0625+0.0625) \\ &= 2.79 \ \text{ bits/symbol}\end{aligned}$$

$$\begin{aligned}H &= 0.25\log_2\left(\frac{1}{0.25}\right)+0.21\log_2\left(\frac{1}{0.21}\right)+0.15\log_2\left(\frac{1}{0.15}\right) \\ &\quad +0.14\log_2\left(\frac{1}{0.14}\right)+(4)(0.0625)\log_2\left(\frac{1}{0.0625}\right) \\ &\approx 2.78\end{aligned}$$

Property of Huffman code

- $H \leq$ average length of Huffman code $\leq H + 1$
- Not unique – but all result in same average length
- Code words uniquely decodable

Efficient coding of a random message

- Example

  You would like to set up your own telephone system that connects you to your three best friends

  The question is how to design efficient binary phone numbers

Friend	Probability	Phone number					
Alice	1/4	0011	001101	0	00	0	10
Bob	1/2	0011	001110	1	11	11	0
Carol	1/4	1100	110000	10	10	10	11
		(i)	(ii)	(iii)	(iv)	(v)	(vi)

In this design, Alice and Bob have the same phone number

The system obviously will not be able to connect properly

Friend	Probability	Phone number					
Alice	1/4	0011	001101	0	00	0	10
Bob	1/2	0011	001110	1	11	11	0
Carol	1/4	1100	110000	10	10	10	11
		(i)	(ii)	(iii)	(iv)	(v)	(vi)

This is much better, i.e. the code will actually work.

However, the phone numbers are quite long and therefore the design is rather inefficient

Friend	Probability	Phone number					
Alice	1/4	0011	001101	0	00	0	10
Bob	1/2	0011	001110	1	11	11	0
Carol	1/4	1100	110000	10	10	10	11
		(i)	(ii)	(iii)	(iv)	(v)	(vi)

- Much shorter and, at the same time, we have made sure that we do not use the same codeword twice
- System will not work
- Code **<u>is not uniquely decodable</u>**
- if you dial 10 this could mean “Carol” or also “Bob, Alice.”
- Or, in other words, the telephone system will never connect you to Carol, because once you dial 1, it will immediately connect you to Bob

Uniquely decodable

Friend	Probability	Phone number					
Alice	1/4	0011	001101	0	00	0	10
Bob	1/2	0011	001110	1	11	11	0
Carol	1/4	1100	110000	10	10	10	11
		(i)	(ii)	(iii)	(iv)	(v)	(vi)

Quite efficient code that is functional
But we note something: when calling Alice, why do we have to dial two zeros?
After the first zero it is already clear to whom we would like to be connected

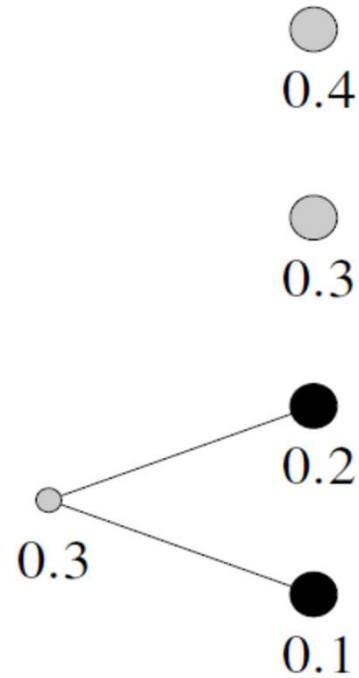
An algorithm - Huffman coding

- **<u>Step 1:</u>** Create r leaves corresponding to the r possible symbols and assign their probabilities $p_1, \ldots, p_r$. Mark these leaves as active
- **<u>Step 2:</u>** Create a new node that has the two *least likely* active leaves or nodes as children. Activate this new node and deactivate its children
- **<u>Step 3:</u>** If there is only one active node left, root it. Otherwise, go to Step 2

Huffman – example

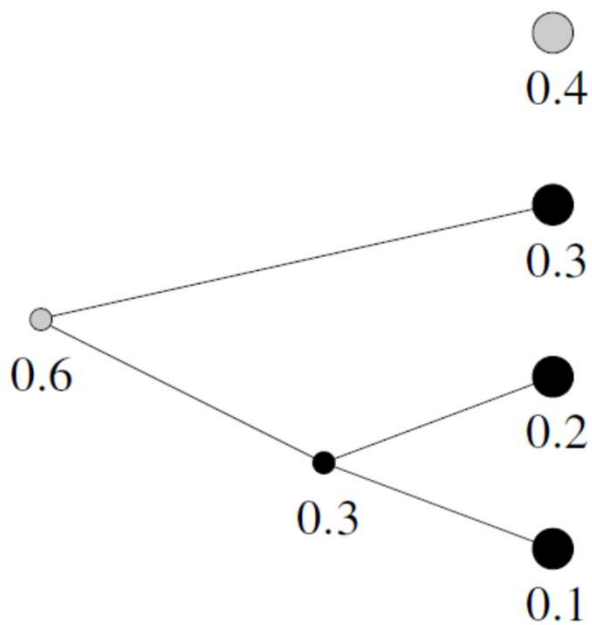
- P_1=0.4, P_2=0.3, P_3=0.2, P_4=0.1

Step 1

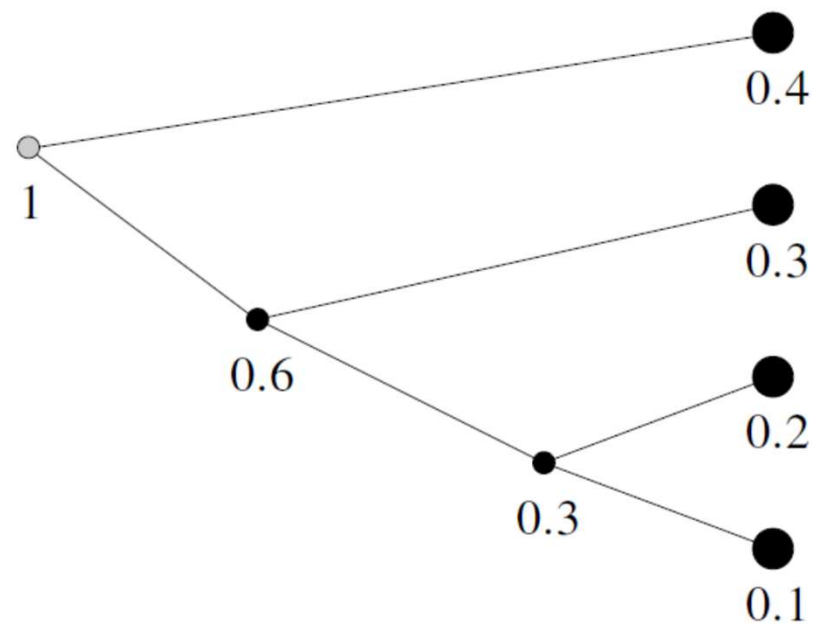


Step 2, first time

Huffman – example

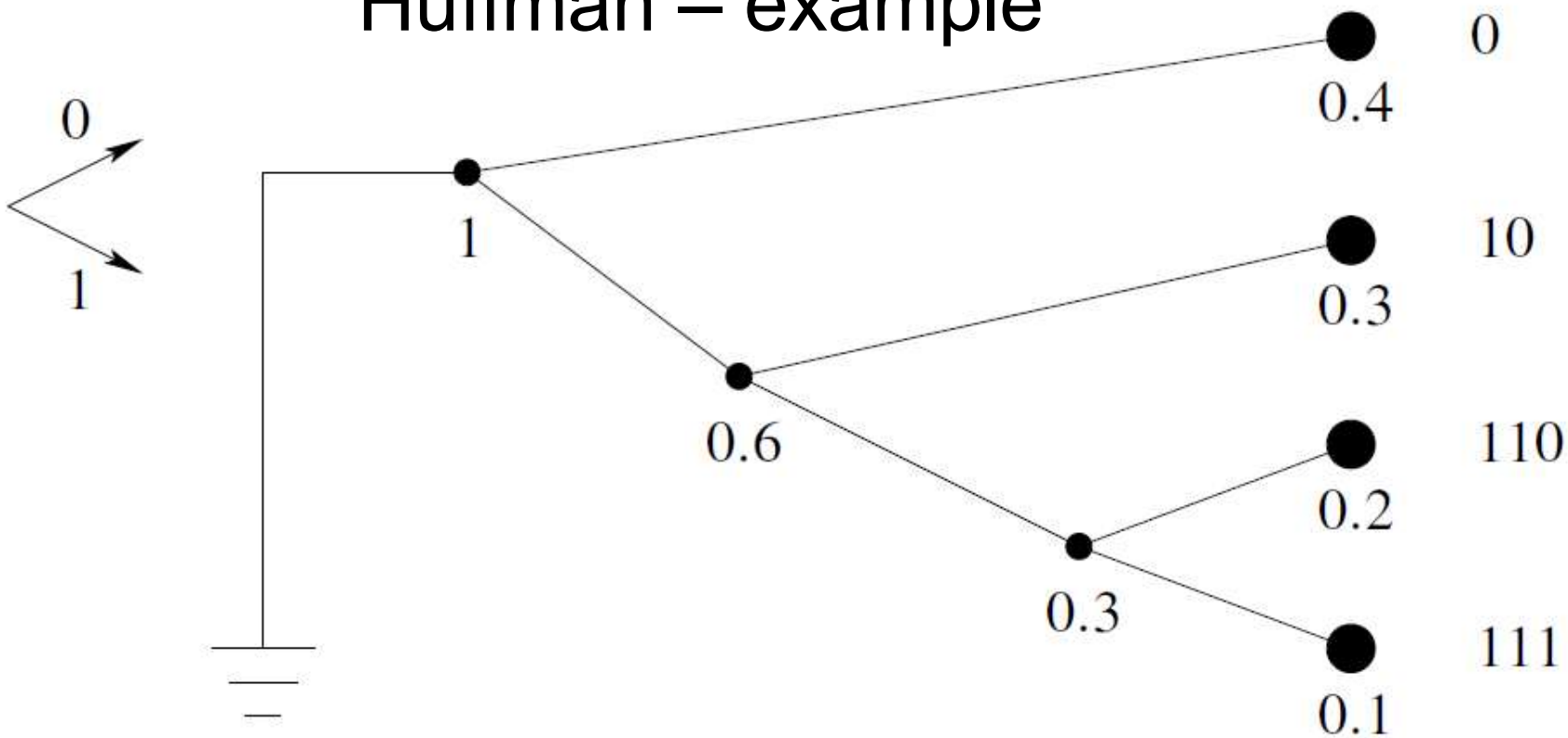


Step 2, second time



Step 2, third time

Huffman – example

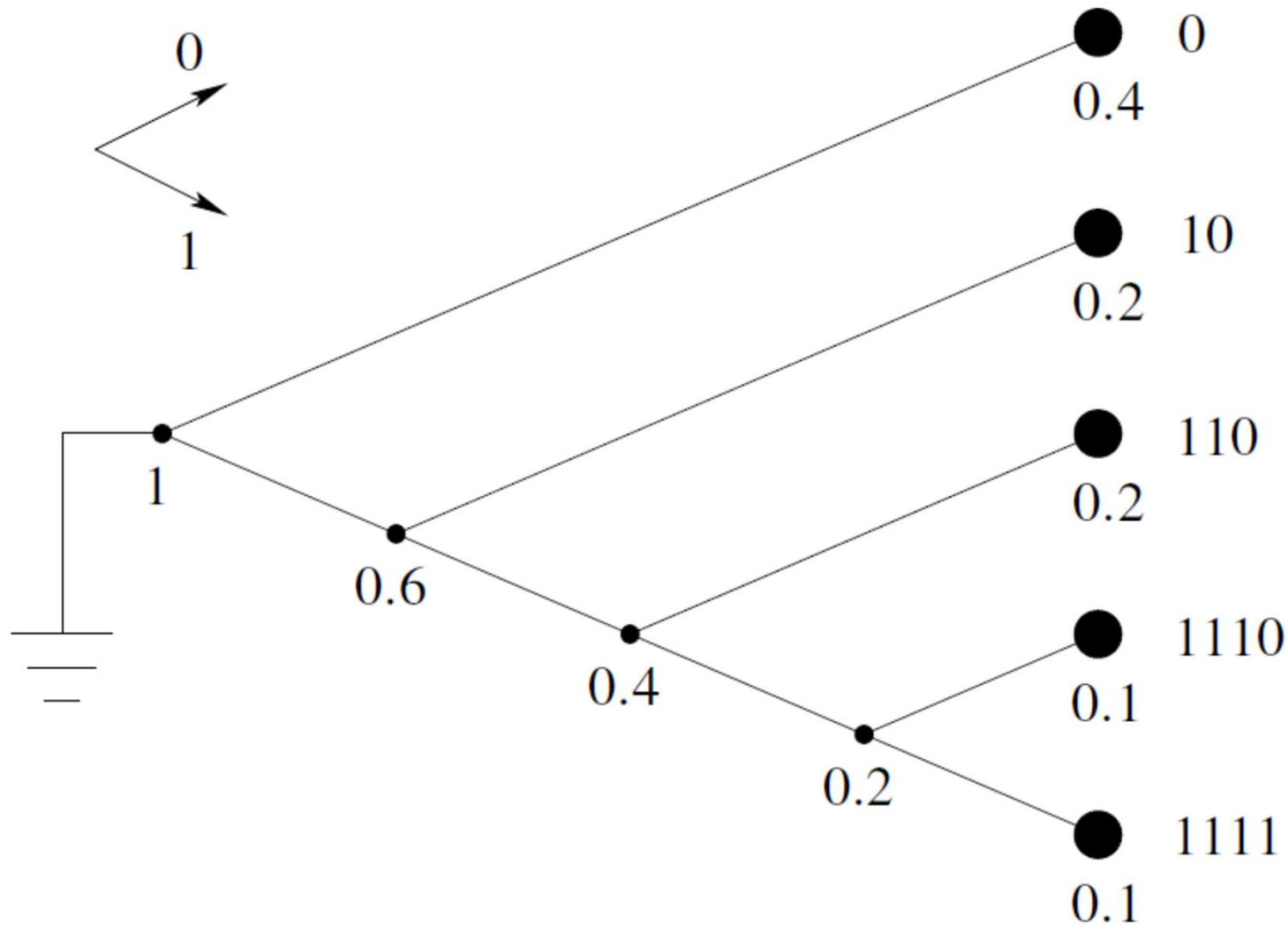


Every time you travel in a branch append 0 or 1 depending upon direction

$$L_{av} = 0.4*1 + 0.3*2 + 0.2*3 + 0.1*3 = 1.9$$

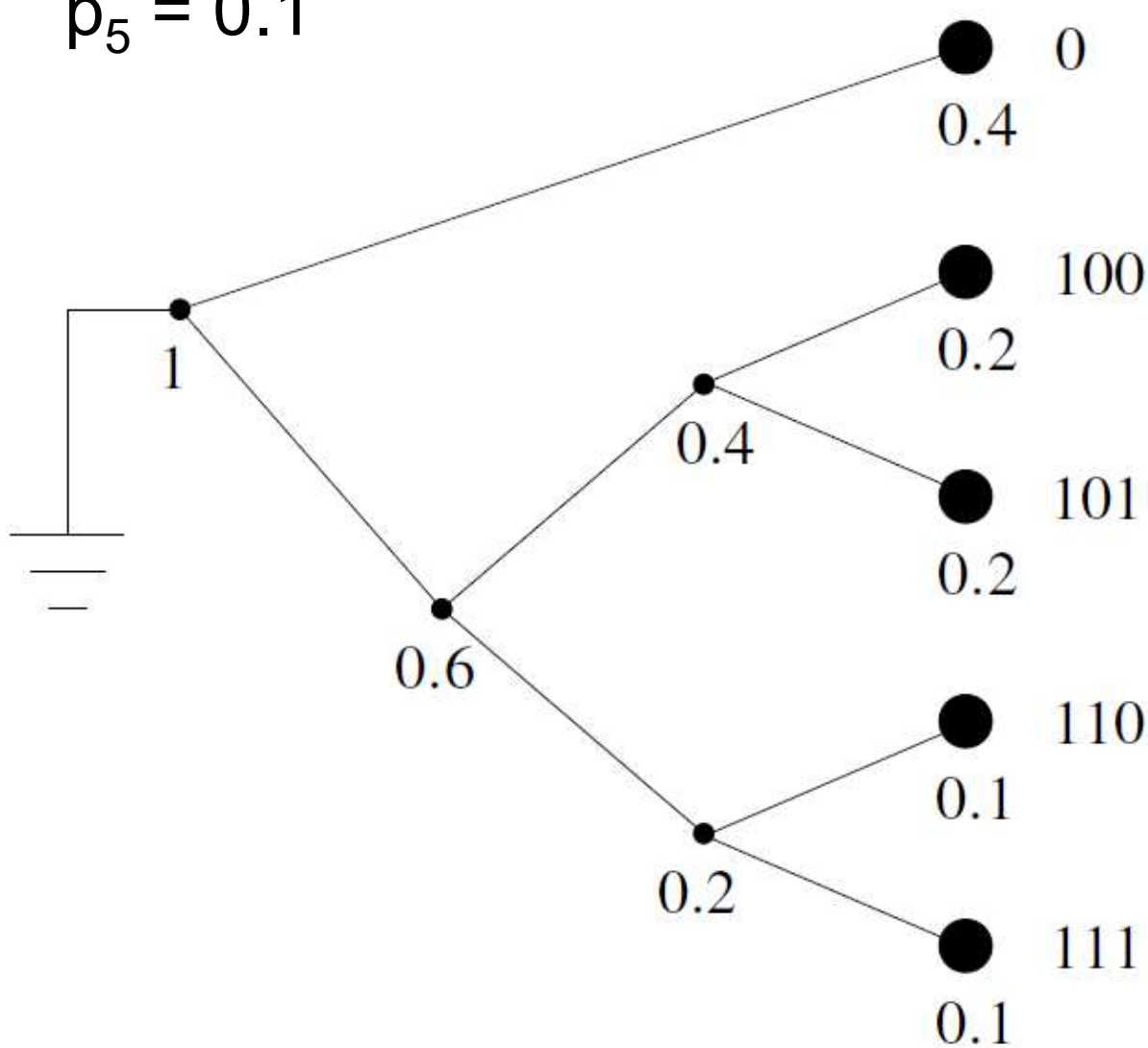
Huffman code for the given PDF

- $p_1 = 0.4$, $p_2 = 0.2$, $p_3 = 0.2$, $p_4 = 0.1$, $p_5 = 0.1$



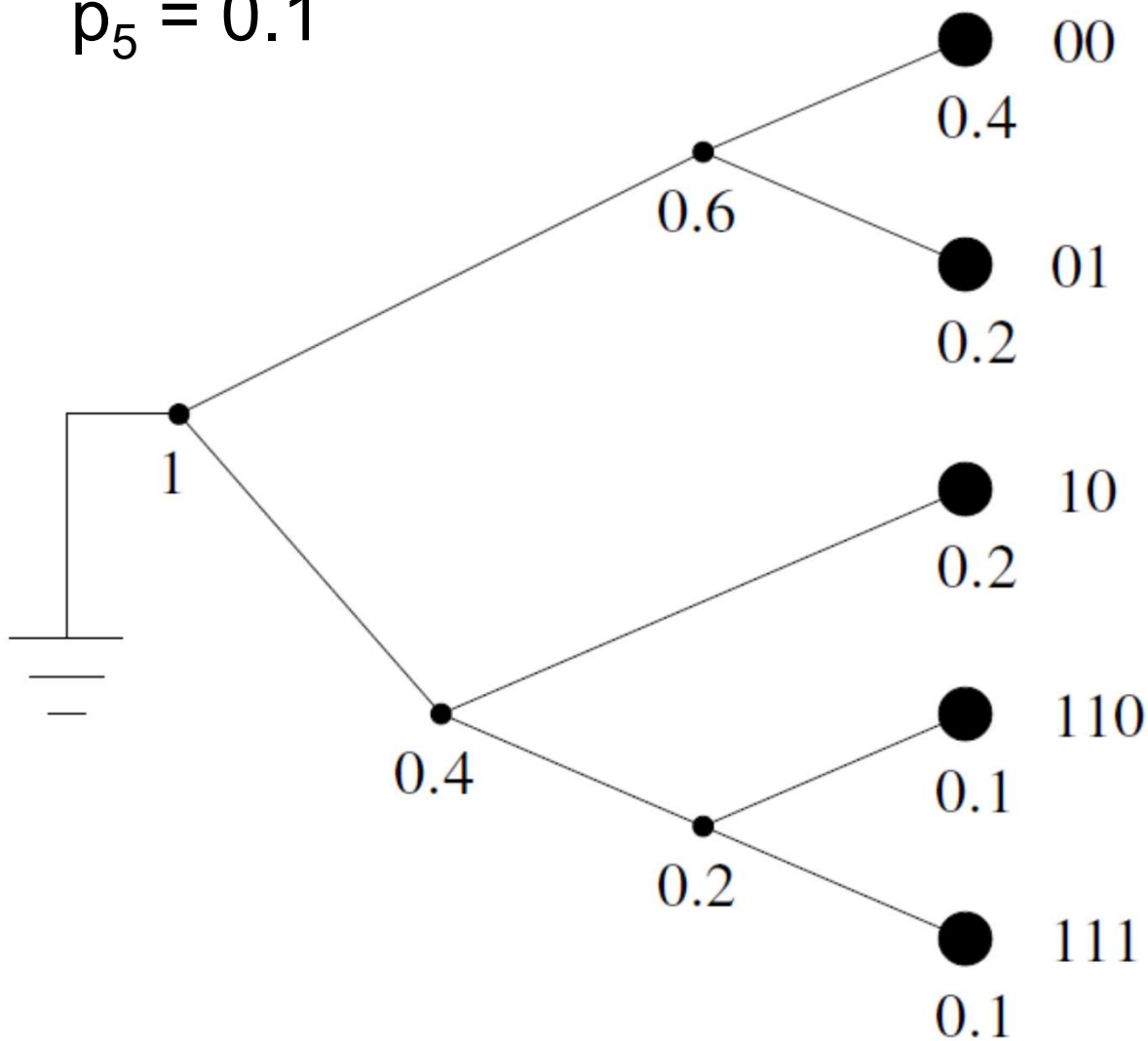
- $p_1 = 0.4$, $p_2 = 0.2$, $p_3 = 0.2$, $p_4 = 0.1$, $p_5 = 0.1$

Huffman code for the same PDF (previous slide)



- $p_1 = 0.4$, $p_2 = 0.2$, $p_3 = 0.2$, $p_4 = 0.1$, $p_5 = 0.1$

Huffman code for the same PDF (previous slide)



Average length of all versions

Version 1:

$L_{av} = 0.4*1 + 0.2*2 + 0.2*3 + 0.1*4 + 0.1*4 = 2.2$

Version 2:

$L_{av} = 0.4*1 + 0.2*3 + 0.2*3 + 0.1*3 + 0.1*3 = 2.2$

Version 3:

$L_{av} = 0.4*2 + 0.2*2 + 0.2*2 + 0.1*3 + 0.1*3 = 2.2$

- Huffman Implementations not unique
- Always results in same average length